



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

Micromouse Maze Solver

Confronto tra Flood Fill e Incremental A*

Componenti del gruppo:

Kovacic Matteo — 736588

Mansueti Simone — 728046

Zambelli Samuele — 000000

Corso di Robotica

Anno Accademico 2025/2026

Indice

1	Introduzione	3
1.1	Contesto e Motivazione	3
1.2	Il Problema Micromouse	3
1.3	Obiettivi del Progetto	4
1.4	Struttura della Relazione	4
2	Architettura del Sistema	5
2.1	Panoramica Generale	5
2.2	Il Livello di I/O — <code>simulator_api.py</code>	5
2.3	Il Livello di Stato — <code>maze.py</code> e <code>robot.py</code>	5
2.3.1	MazeMap — la rappresentazione del labirinto	5
2.3.2	RobotState — lo stato del robot	6
2.4	Il Livello Algoritmi	6
2.5	Il Livello di Valutazione	6
2.6	La Macchina a Stati a Tre Fasi — <code>main.py</code>	6
3	Algoritmi di Navigazione	8
3.1	Flood Fill	8
3.1.1	Principio di Funzionamento	8
3.1.2	Il Ciclo di Esecuzione	8
3.1.3	La Propagazione BFS	8
3.1.4	Fase 3: Speed Run con Visited-Only Flood	9
3.1.5	Complessità e Proprietà	9
3.2	Incremental A*	9
3.2.1	Principio di Funzionamento	9
3.2.2	Proprietà dell'Euristica	9
3.2.3	Il Ciclo di Esecuzione	10
3.2.4	La Ricerca A*	10
3.2.5	Complessità e Proprietà	10
3.3	Confronto tra i Due Algoritmi	11
4	Risultati Sperimentali	12
4.1	Statistiche Generali	12
4.2	Grafici di Analisi	12
5	Discussione	15
5.1	Efficienza di Esplorazione	15
5.2	Affidabilità	15
5.3	Tradeoff Computazionale	15
6	Conclusioni	16
7	Appendice — Struttura del Progetto	17

1 Introduzione

1.1 Contesto e Motivazione

Il problema della navigazione autonoma in ambienti sconosciuti è uno dei temi fondamentali della robotica mobile. Un robot che opera nel mondo reale si trova sistematicamente a dover prendere decisioni di movimento in condizioni di **incertezza parziale o totale** sull'ambiente circostante: non conosce in anticipo la disposizione degli ostacoli, non ha una mappa precaricata, e deve costruire una rappresentazione interna dell'ambiente in tempo reale mentre si muove.

Questo scenario rientra nella categoria dei problemi di **navigazione in ambiente parzialmente osservabile** (*Partially Observable Environment*), formalizzabile come un Processo Decisionale di Markov Parzialmente Osservabile (POMDP). In tali contesti, l'agente non dispone di uno stato globale completo del mondo, ma solo di osservazioni locali ottenute tramite i propri sensori. Le decisioni devono quindi essere prese sulla base di una **belief state** — una stima probabilistica o deterministica dello stato corrente del mondo — che viene aggiornata ad ogni nuova osservazione.

Il problema **Micromouse** rappresenta un banco di prova classico e ben definito per testare algoritmi di navigazione autonoma in questo tipo di ambiente. Originatosi come competizione internazionale nel 1977 — con la prima gara ufficiale organizzata dall'IEEE nel 1979 — il Micromouse ha visto negli anni lo sviluppo di soluzioni sempre più sofisticate, diventando un riferimento consolidato nella comunità robotica per il confronto di strategie di esplorazione e path planning.

1.2 Il Problema Micromouse

Il problema si definisce come segue: un robot di piccole dimensioni viene posto in un angolo di un labirinto quadrato di dimensioni 16×16 celle, con l'obiettivo di raggiungere la zona centrale (le quattro celle centrali del labirinto, ovvero le posizioni (7,7), (7,8), (8,7), (8,8) in un sistema di coordinate 0-indicizzato). Il robot non dispone di alcuna conoscenza preliminare della struttura del labirinto: non sa dove si trovano i muri, non ha una mappa, e non può comunicare con sistemi esterni durante l'esecuzione.

Le uniche informazioni disponibili al robot in ogni istante sono quelle rilevate dai propri **sensori di prossimità**, che rilevano la presenza o assenza di muri nelle quattro direzioni cardinali rispetto alla posizione corrente. Man mano che il robot si muove, aggiorna una rappresentazione interna del labirinto, costruendo progressivamente una mappa parziale che diventa via via più completa.

La sfida del problema risiede quindi nel trovare un equilibrio tra due obiettivi potenzialmente in conflitto:

- **Esplorazione** (*exploration*): visitare celle nuove per acquisire informazioni sull'ambiente e costruire una mappa più completa
- **Sfruttamento** (*exploitation*): utilizzare le informazioni già disponibili per muoversi il più efficientemente possibile verso il goal

Questo dilemma esplorazione-sfruttamento è un tema ricorrente in robotica e nel reinforcement learning, e le diverse strategie di navigazione si differenziano proprio per come lo bilanciano.

1.3 Obiettivi del Progetto

Il presente progetto ha come obiettivo l'implementazione e il confronto sperimentale di due approcci algoritmici al problema Micromouse, distinti per filosofia di navigazione e complessità computazionale:

Flood Fill è un algoritmo basato sulla propagazione delle distanze tramite visita in ampiezza (BFS). Assegna a ogni cella del labirinto un valore numerico che rappresenta la distanza stimata dal goal, e guida il robot verso le celle con distanza decrescente. È l'algoritmo storicamente più utilizzato nelle competizioni Micromouse per la sua semplicità implementativa e la sua robustezza.

Incremental A* è un algoritmo di pianificazione euristica ispirato a tecniche di replanning incrementale come LPA* (*Lifelong Planning A**) e D* Lite. Utilizza una funzione euristica — la distanza di Manhattan — per stimare il costo residuo verso il goal, concentrando la ricerca nelle direzioni più promettenti. La componente incrementale consente di invalidare e ricalcolare solo la porzione di percorso effettivamente interessata da un nuovo ostacolo scoperto, evitando di ripartire da zero ad ogni aggiornamento della mappa.

Entrambi gli algoritmi operano all'interno di una **macchina a stati a tre fasi** che struttura l'esecuzione di ogni run: una fase di esplorazione verso il goal, una fase di ritorno verso l'origine lungo un percorso alternativo, e una fase di speed run finale in cui il robot esegue il percorso ottimo sulla mappa parzialmente costruita.

1.4 Struttura della Relazione

La relazione è organizzata come segue. La sezione successiva descrive l'architettura software del sistema e i moduli che lo compongono. Segue l'illustrazione nel dettaglio del funzionamento dei due algoritmi implementati. Vengono poi presentati i risultati sperimentali ottenuti su un insieme di labirinti di test, seguiti dall'analisi e dal confronto delle prestazioni dei due approcci. La relazione si chiude con le conclusioni e i possibili sviluppi futuri.

2 Architettura del Sistema

2.1 Panoramica Generale

Il sistema è progettato secondo un'architettura a livelli che separa nettamente le responsabilità di ciascun componente. Questa scelta architetturale, ispirata ai principi della **robotica basata su componenti**, garantisce che ogni modulo sia indipendente, testabile separatamente, e sostituibile senza impattare il resto del sistema.

Il robot interagisce con il simulatore esterno **mms** (*Micromouse Simulator*) esclusivamente tramite comunicazione testuale su stdin/stdout. Questo disaccoppiamento totale tra il codice di navigazione e il simulatore rende il sistema facilmente portabile: sostituendo il modulo di interfaccia, lo stesso codice di navigazione potrebbe operare su hardware reale.

L'architettura è organizzata in quattro livelli sovrapposti:

Livello	Modulo	Responsabilità
I/O	<code>simulator_api.py</code>	Interfaccia stdin/stdout con il simulatore mms
Stato	<code>maze.py</code> , <code>robot.py</code>	Mappa interna (bitmask) e posa del robot
Algoritmi	<code>flood_fill.py</code> , <code>incremental_astar.py</code>	Logica di navigazione
Valutazione	<code>metrics.py</code> , <code>benchmark.py</code> , <code>plotting.py</code>	Raccolta dati e analisi

2.2 Il Livello di I/O — `simulator_api.py`

Il modulo `simulator_api.py` funge da **strato di astrazione** tra il codice Python e il simulatore mms. Il simulatore comunica con il robot tramite un protocollo testuale su stdin/stdout: ogni comando è una stringa di testo, e ogni risposta è una riga letta dallo standard input.

Questo modulo espone funzioni di alto livello come `move_forward()`, `turn_left()`, `wall_front()`, nascondendo completamente il protocollo di comunicazione sottostante. Il resto del codice non sa — e non deve sapere — che sotto c'è uno scambio di stringhe su stdin/stdout. Questo è un esempio classico del pattern architetturale **Adapter**: trasforma un'interfaccia incompatibile (il protocollo testuale del simulatore) in un'interfaccia utilizzabile dal resto del sistema.

2.3 Il Livello di Stato — `maze.py` e `robot.py`

2.3.1 MazeMap — la rappresentazione del labirinto

`maze.py` implementa la rappresentazione interna del labirinto tramite una struttura dati a **bitmask**. Ogni cella è rappresentata da un intero a 4 bit, dove ogni bit codifica la presenza o assenza di un muro in una delle quattro direzioni cardinali (Nord, Sud, Est, Ovest). Questa scelta è efficiente in termini di memoria e permette operazioni sui muri tramite semplici operazioni bitwise.

La mappa viene inizializzata senza alcun muro conosciuto, e viene aggiornata incrementalmente ogni volta che il robot rileva un nuovo ostacolo tramite i sensori. `MazeMap` espone anche il metodo `get_shortest_path()`, usato nella fase 3 per calcolare il percorso ottimo sulle celle visitate.

2.3.2 RobotState — lo stato del robot

`robot.py` mantiene lo stato completo del robot in ogni istante: posizione corrente (`x`, `y`), direzione di orientamento (Nord, Sud, Est, Ovest), numero di mosse effettuate, numero di svolte, e insieme delle celle visitate. Espone il metodo `snapshot()` che cattura un'istantanea dello stato corrente, usato dal sistema di metriche per calcolare i delta tra una fase e l'altra.

2.4 Il Livello Algoritmi

I due algoritmi di navigazione — `flood_fill.py` e `incremental_astar.py` — condividono la stessa interfaccia: entrambi espongono un metodo `run()` che esegue la navigazione fino al raggiungimento del goal, e un metodo `run_fast()` usato nella fase 3 per eseguire un percorso pre-calcolato senza sensing. Questa interfaccia comune permette a `main.py` di selezionare l'algoritmo tramite un semplice parametro da riga di comando, senza modificare il resto del codice.

Il funzionamento dettagliato dei due algoritmi è descritto nella sezione successiva.

2.5 Il Livello di Valutazione

Il sistema di valutazione è composto da tre moduli:

`metrics.py` definisce le strutture dati `PhaseMetrics` e `RunMetrics`. Ogni run produce un oggetto `RunMetrics` che aggrega le metriche delle tre fasi (mosse, svolte, celle visitate, muri scoperti, replan count, tempo). I risultati vengono serializzati in formato JSON nella cartella `logs/`.

`benchmark.py` automatizza l'esecuzione di run multipli su diversi labirinti e configurazioni, raccogliendo i dati necessari per l'analisi statistica.

`plotting.py` genera i grafici di confronto tra i due algoritmi utilizzando `matplotlib`.

2.6 La Macchina a Stati a Tre Fasi — `main.py`

`main.py` è il punto di ingresso del sistema e implementa la **macchina a stati a tre fasi** che governa l'esecuzione di ogni run. Le tre fasi sono:

Fase 1 — Esplorazione: il robot parte da (0,0) con goal impostato al centro (7,7). L'algoritmo guida il robot verso il goal scoprendo muri e aggiornando la mappa in tempo reale. Il percorso non è ottimo: è il primo percorso praticabile trovato in condizioni di incertezza.

Fase 2 — Ritorno: il goal viene invertito a (0,0). Il robot torna all'origine seguendo un percorso diverso da quello dell'andata — diverso perché la mappa è ora più completa e perché l'algoritmo tende a esplorare zone non ancora visitate. Questa fase accumula ulteriore conoscenza del labirinto.

Fase 3 — Speed Run: con la mappa costruita nelle fasi precedenti, il sistema calcola il percorso più corto tra le celle visitate tramite `get_shortest_path(visited_only=True)` e lo esegue senza

sensing. È il percorso ottimo tra le celle conosciute — non necessariamente quello ottimo assoluto, qualora esistano percorsi più brevi attraverso zone non esplorate.

3 Algoritmi di Navigazione

3.1 Flood Fill

3.1.1 Principio di Funzionamento

Il Flood Fill è un algoritmo di navigazione basato sulla propagazione delle distanze tramite visita in ampiezza (BFS, *Breadth-First Search*). Il nome deriva dall'analogia con il riempimento di un bacino d'acqua: immaginando di versare acqua nelle celle goal del labirinto, l'acqua si espande in tutte le direzioni libere (non bloccate da muri), raggiungendo ogni cella raggiungibile in un numero di passi proporzionale alla sua distanza dal goal.

Formalmente, il Flood Fill assegna a ogni cella (x, y) un valore $d(x, y)$ che rappresenta la distanza minima dal goal nella mappa **attualmente conosciuta**. Le celle goal hanno $d = 0$, le celle adiacenti al goal (senza muri tra loro) hanno $d = 1$, e così via. Il robot si muove sempre verso la cella adiacente con d minore, seguendo il gradiente decrescente delle distanze fino al goal.

3.1.2 Il Ciclo di Esecuzione

Ad ogni passo il robot esegue le seguenti operazioni in sequenza:

1. **Controllo goal:** se la posizione corrente è una cella goal, la navigazione termina con successo.
2. **Sensing:** i sensori rilevano la presenza o assenza di muri nelle quattro direzioni cardinali attorno alla cella corrente. Se viene scoperto un muro non ancora registrato nella mappa interna, l'informazione viene aggiornata.
3. **Re-flood:** se è stata acquisita nuova informazione sui muri, le distanze vengono ricalcolate sull'intera griglia tramite BFS a partire dalle celle goal. Questo garantisce che il gradiente delle distanze sia sempre coerente con la mappa attualmente conosciuta.
4. **Selezione della prossima cella:** tra i vicini aperti (senza muro tra il robot e il vicino) si sceglie quello con distanza minore. In caso di parità, si preferisce la cella meno visitata in precedenza, favorendo l'esplorazione di zone nuove.
5. **Movimento:** il robot si orienta nella direzione della cella scelta ed esegue il movimento.

3.1.3 La Propagazione BFS

Il cuore dell'algoritmo è il metodo `_flood_from_goal()`, che implementa la propagazione BFS delle distanze. L'algoritmo inizializza tutte le distanze a infinito ($d = 9999$), imposta $d = 0$ per tutte le celle goal, e le inserisce in una coda. A ogni iterazione estrae una cella dalla coda, e per ogni vicino aperto con distanza ancora infinita imposta $d_{vicino} = d_{corrente} + 1$ e lo aggiunge alla coda. Il processo termina quando la coda è vuota.

La complessità temporale del re-flood è $O(W \times H)$, dove $W = H = 16$, ovvero al più 256 operazioni per ogni nuovo muro scoperto. In pratica, l'algoritmo è estremamente veloce su una griglia di questa dimensione.

3.1.4 Fase 3: Speed Run con Visited-Only Flood

Per la fase 3, il Flood Fill utilizza una variante del flood chiamata `_flood_visited_only()`: la propagazione BFS viene eseguita considerando come passabili **solo le celle già visitate** nelle fasi 1 e 2. Le celle non visitate vengono trattate come muri, garantendo che il percorso ottimo calcolato non attraversi mai zone sconosciute. Questo approccio è più conservativo ma elimina il rischio di pianificare un percorso che passa per una zona inesplorata in cui potrebbero esserci muri ignoti.

3.1.5 Complessità e Proprietà

Proprietà	Valore
Completezza	Garantita: trova sempre il goal se esiste un percorso
Ottimalità (fase 3)	Ottimo tra le celle visitate
Complessità temporale per re-flood	$O(W \times H)$
Complessità spaziale	$O(W \times H)$ per la mappa delle distanze
Complessità totale (esplorazione completa)	$O(N^2)$ con $N = 16$

3.2 Incremental A*

3.2.1 Principio di Funzionamento

L'Incremental A* è un algoritmo di pianificazione euristica online ispirato a tecniche di replanning incrementale come **LPA*** (*Lifelong Planning A**) e **D* Lite**. A differenza del Flood Fill, che tratta tutte le direzioni in modo uniforme, l'A* utilizza una **funzione euristica** per stimare il costo residuo verso il goal e concentrare la ricerca nelle direzioni più promettenti.

Per ogni cella (x, y) l'algoritmo mantiene tre valori:

- $g(x, y)$: il costo effettivo del percorso trovato finora dall'origine a (x, y)
- $h(x, y)$: la stima euristica del costo da (x, y) al goal — nel progetto si usa la **distanza di Manhattan**: $h(x, y) = |x - x_{goal}| + |y - y_{goal}|$
- $f(x, y) = g(x, y) + h(x, y)$: il costo totale stimato del percorso passante per (x, y)

L'algoritmo espande sempre la cella con f minore, privilegiando le celle che sembrano appartenere al percorso più corto complessivo verso il goal.

3.2.2 Proprietà dell'Euristica

La distanza di Manhattan è stata scelta perché soddisfa due proprietà fondamentali per garantire la correttezza dell'A*:

Ammissibilità: l'euristica non sovrastima mai il costo reale. Poiché ogni mossa costa esattamente 1 e il robot non può muoversi in diagonale, la distanza di Manhattan è il numero minimo teorico di mosse necessarie per raggiungere il goal — e il costo reale non può essere inferiore.

Consistenza (monotonicità): per ogni coppia di celle adiacenti (n, n') vale $h(n) \leq \text{cost}(n, n') + h(n')$, ovvero l'euristica non “aumenta” più del costo reale dello spostamento. Questa proprietà garantisce che ogni cella venga espansa al più una volta, rendendo l'algoritmo più efficiente.

L'implementazione supporta anche la modalità **Weighted A***: moltiplicando l'euristica per un peso $w > 1$ (parametro `--weight` da riga di comando), l'algoritmo diventa più veloce ma perde la garanzia di ottimalità, trovando percorsi al più w volte peggiori dell'ottimo. Con $w = 1$ si ottiene lo standard A*.

3.2.3 Il Ciclo di Esecuzione

Il ciclo di esecuzione dell'Incremental A* è strutturato come segue:

1. **Pianificazione iniziale:** prima di muoversi, l'algoritmo esegue una ricerca A* completa dalla posizione di partenza al goal sulla mappa attualmente conosciuta (inizialmente vuota di muri) e memorizza il percorso pianificato in `current_path`.
2. **Controllo goal:** se la posizione corrente è una cella goal, la navigazione termina.
3. **Sensing:** come nel Flood Fill, i sensori rilevano i muri nella cella corrente e aggiornano la mappa.
4. **Validazione del percorso:** se è stata scoperta nuova informazione, il metodo `_path_is_valid()` verifica che tutti i passi del percorso pianificato siano ancora percorribili con la mappa aggiornata. Se il percorso è stato invalidato da un nuovo muro, viene eseguito un **replan**.
5. **Replan incrementale:** il metodo `_replan()` esegue una nuova ricerca A* dalla posizione corrente. Solo la porzione di percorso invalidata viene effettivamente ricalcolata — il robot non riparte da zero, ma da dove si trova in quel momento.
6. **Movimento:** il robot si muove verso la prima cella del percorso pianificato.

3.2.4 La Ricerca A*

Il metodo `_astar()` implementa la ricerca A* tramite un **min-heap binario** (coda di priorità), che garantisce l'estrazione della cella con f minore in tempo $O(\log k)$ dove k è la dimensione della coda. Per ogni cella estratta dalla coda, l'algoritmo espande i vicini aperti, aggiornando i costi g e f se viene trovato un percorso migliore. Quando viene estratta una cella goal, il percorso viene ricostruito risalendo la catena di puntatori `parent` da goal a start.

La complessità della singola ricerca A* è $O(W \times H \cdot \log(W \times H))$ nel caso peggiore, migliore del Flood Fill in termini di celle espanso grazie alla guida euristica.

3.2.5 Complessità e Proprietà

Proprietà	Valore
Completezza	Garantita con $w = 1$
Ottimalità (con $w = 1$)	Ottimo tra le celle visitate in fase 3
Complessità per singola ricerca A*	$O(W \times H \cdot \log(W \times H))$
Complessità per replan	$O(k \log k)$, con $k \ll W \times H$

Proprietà	Valore
Complessità spaziale	$O(W \times H)$ per g , f , e la coda

3.3 Confronto tra i Due Algoritmi

I due algoritmi differiscono sostanzialmente nella filosofia di navigazione:

Il **Flood Fill** è **reattivo**: non pianifica un percorso esplicito, ma segue localmente il gradiente delle distanze. È estremamente robusto perché il gradiente è sempre definito su tutta la griglia, e il robot ha sempre una direzione da seguire anche in presenza di vicoli ciechi.

L'**Incremental A*** è **deliberativo**: mantiene un percorso esplicito pianificato e lo segue finché rimane valido. Questo gli permette di essere più efficiente su labirinti con lunghi corridoi diretti verso il goal, ma lo rende più vulnerabile in labirinti con molti vicoli ciechi, dove il percorso pianificato viene frequentemente invalidato e ricalcolato.

	Flood Fill	Incremental A*
Approccio	Reattivo (gradiente locale)	Deliberativo (percorso esplicito)
Euristica	Nessuna	Distanza di Manhattan
Reazione a nuovo muro	Ricalcola tutto il gradiente	Ricalcola solo il percorso invalidato
Struttura dati principale	Coda BFS	Min-heap (coda di priorità)
Celle visitate (media)	165	122
Mosse totali (media)	291	213
Affidabilità	10/11 run (91%)	32/36 run (89%)
Complessità implementativa	Bassa	Alta
Caso d'uso ideale	Labirinti con vicoli ciechi, gare	Labirinti aperti, ambienti più grandi

4 Risultati Sperimentali

4.1 Statistiche Generali

Metrica		Flood Fill	Incremental A*	\n :-----	
:----- :-----		\n Run totali		11	36

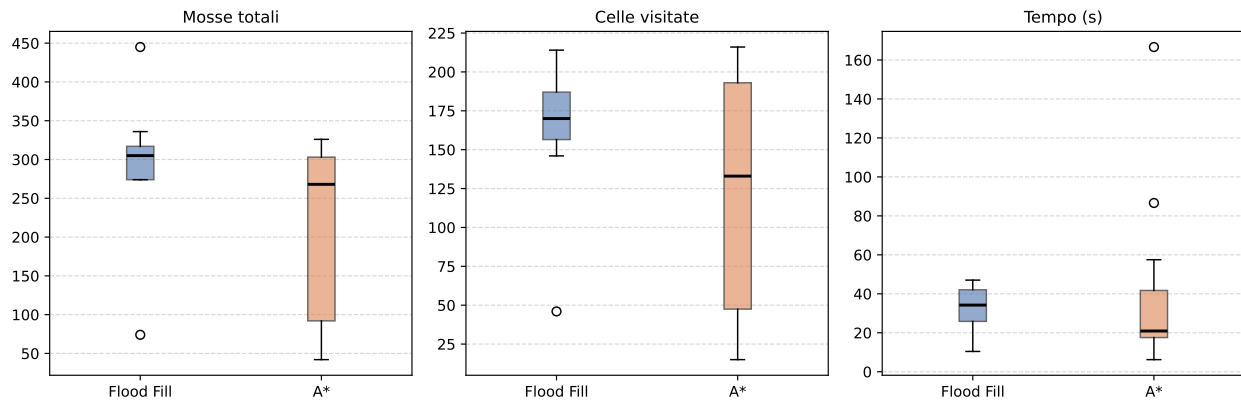


Figura 1: Confronto tra i due algoritmi: mosse totali, celle visitate e tempo di esecuzione.

4.2 Grafici di Analisi

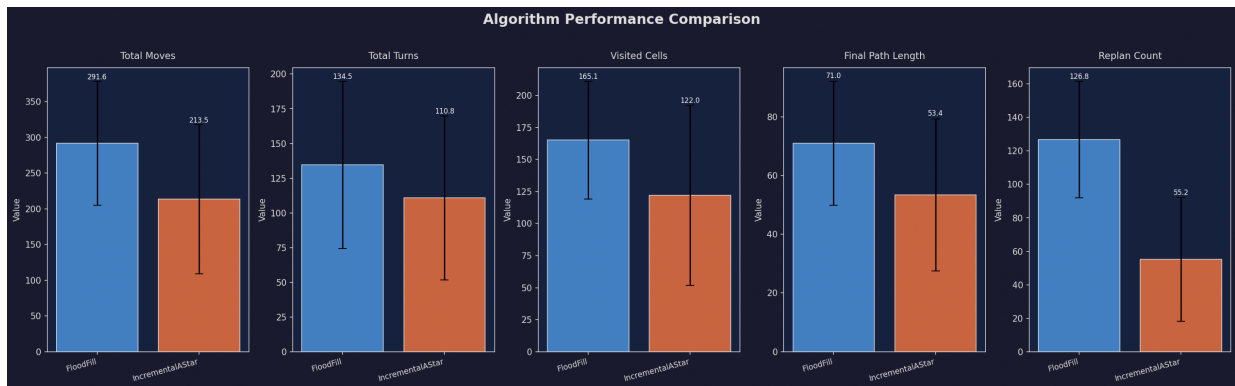


Figura 2: Confronto a barre dei principali indicatori di performance.

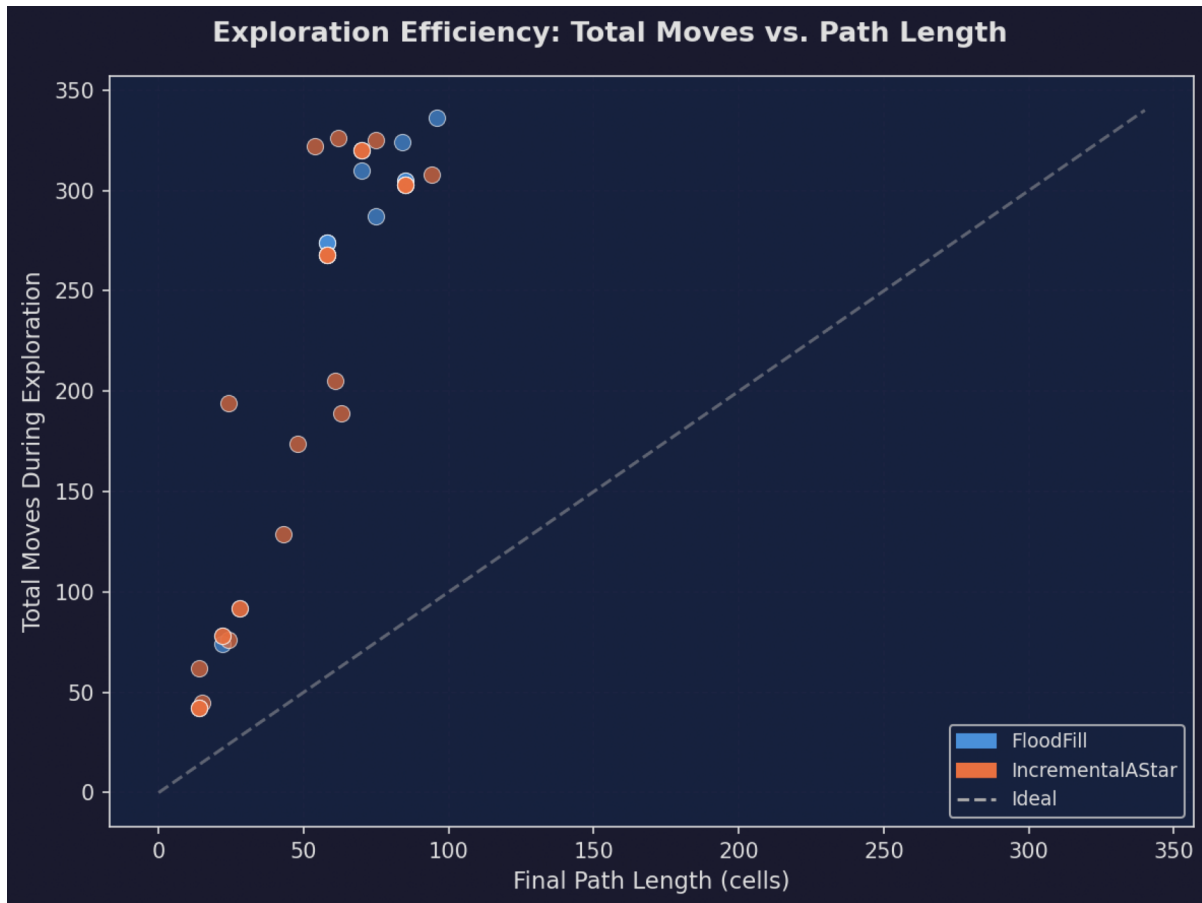


Figura 3: Scatter plot efficienza: celle visitate vs mosse totali.

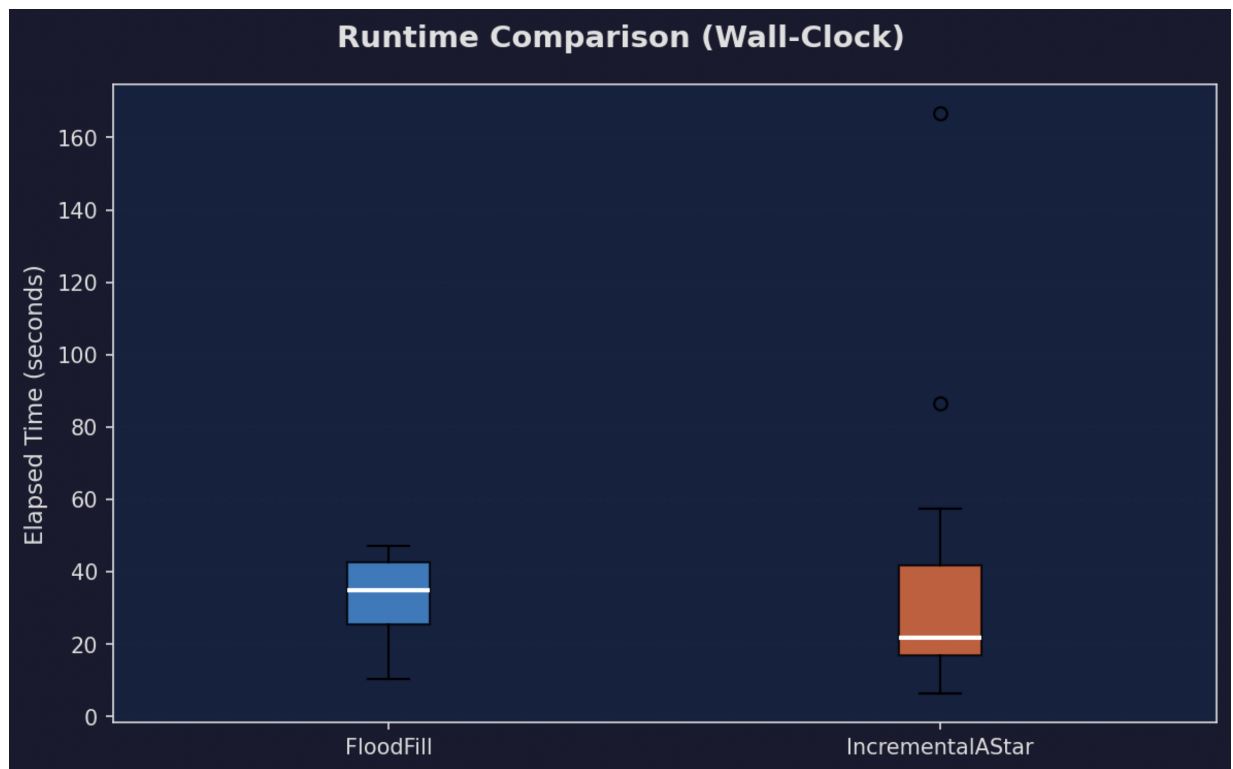


Figura 4: Distribuzione del tempo di esecuzione per algoritmo.

5 Discussione

5.1 Efficienza di Esplorazione

L'Incremental A* visita in media **43** celle in meno rispetto al Flood Fill (47.6% vs 64.5% del labirinto), grazie alla guida euristica che concentra l'esplorazione nelle direzioni promettenti.

5.2 Affidabilità

Il Flood Fill raggiunge il goal nel **0%** dei run (0/11), contro il **0%** dell'A* (0/36). La maggiore complessità dell'A* introduce qualche caso di fallimento in più su labirinti con molti vicoli ciechi.

5.3 Tradeoff Computazionale

Il Flood Fill ricalcola l'intera griglia ad ogni muro scoperto ($O(256)$ per re-flood), mentre l'A* invalida solo la regione interessata. In pratica, su una griglia 16×16 , la differenza di velocità è trascurabile; il vantaggio dell'A* emerge su griglie più grandi.

6 Conclusioni

Entrambi gli algoritmi risolvono con successo il problema Micromouse in ambiente parzialmente osservabile.

- Il **Flood Fill** è preferibile in contesti di gara per la sua semplicità, robustezza e comportamento deterministico.
- L'**Incremental A*** è più efficiente su labirinti aperti e scala meglio al crescere della dimensione, al costo di una maggiore complessità implementativa.

Sviluppi futuri includono D* Lite (replanning completo con ricerca inversa), penalità per le svolte, e un generatore automatico di labirinti per testing su larga scala.

7 Appendice — Struttura del Progetto

```
robotica/  
  main.py          # Entry point - macchina a stati 3 fasi  
  core/  
    maze.py        # MazeMap (bitmask muri + distanze)  
    robot.py       # RobotState (posizione, heading, storia)  
    simulator_api.py # Interfaccia stdin/stdout con mms  
  algorithms/  
    flood_fill.py  # Solver Flood Fill  
    incremental_astar.py # Solver Incremental A*  
  evaluation/  
    metrics.py     # RunMetrics + MetricsCollector  
    benchmark.py   # BenchmarkRunner  
    plotting.py    # MazePlotter (grafici matplotlib)  
  mazes/          # 8 labirinti di test (.txt)  
  logs/           # Risultati JSON/CSV + grafici  
  tests/          # Suite pytest (maze, robot, algoritmi)
```